



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Tunis El Manar
Institut Supérieur d'Informatique d'El Manar



RAPPORT DE MINI PROJET BIG DATA

Présenté par

Maaoui Siwar

Laadhar Edam

Développement d'une Plateforme d'Analyse Prédictive Immobilière

Table des matières

Introduction générale	1
1 Analyse et Conception du Système	2
1.1 Introduction	3
1.2 Analyse des besoins	3
1.2.1 Diagramme de cas d'utilisation	4
1.3 Architecture globale du système	5
1.4 Pipeline de traitement des données	5
1.5 Modélisation et description des données	6
1.5.1 Présentation du dataset	6
1.5.2 Rôle du dataset dans le système	7
1.6 Fonctionnement du système	8
1.6.1 Interaction utilisateur - système	8
1.6.2 Fonctionnement du système	8
1.6.3 Synthèse	10
1.7 Conclusion	10
2 Implémentation de la Phase d'Ingestion et de Prétraitement des Données	11
2.1 Introduction	12
2.2 Objectif de la phase	12
2.3 Entrées et sorties	12
2.3.1 Entrée	13
2.3.2 Sortie	13
2.4 Technologies utilisées	13
2.5 Architecture de la solution	13
2.5.1 Image Docker	14
2.5.2 Persistance des données avec les volumes Docker	15
2.5.3 Organisation des fichiers	16
2.5.4 Logique de traitement	16
2.6 Réalisation	17
2.7 Conclusion	18

3	Implémentation de la Phase de Stockage Distribué dans HDFS	19
3.1	Introduction	20
3.2	Objectif de la phase	20
3.3	Entrées et sorties	20
3.3.1	Entrée	20
3.3.2	Sortie	20
3.4	Architecture de HDFS	21
3.5	HDFS et le Big Data	21
3.6	Implémentation du mini-cluster HDFS	21
3.6.1	Architecture du cluster	22
3.6.2	Configuration Docker Compose	22
3.6.3	Mapping des ports	23
3.7	Intégration dans le pipeline	23
3.8	Réalisation	23
3.9	Conclusion	24
4	Implémentation de la Phase de Traitement et de Modélisation avec Apache Spark	25
4.1	Introduction	26
4.2	Objectif de la phase	26
4.3	Entrées et sorties	26
4.3.1	Entrée	26
4.3.2	Sortie	26
4.4	Pipeline de traitement	27
4.5	Préparation des données	27
4.6	Modèles de machine learning	28
4.6.1	Régression Linéaire	28
4.6.2	Random Forest	28
4.6.3	Gradient Boosting	28
4.7	Évaluation des performances	28
4.8	Stockage des résultats	28
4.8.1	Table des exécutions (<code>model_runs</code>)	29
4.8.2	Table des prédictions (<code>predictions</code>)	29
4.9	Réalisation	29
4.10	Conclusion	30

5	Implémentation de la Couche de Stockage et d'Exploitation des Données avec PostgreSQL	31
5.1	Introduction	32
5.2	Objectif de la phase	32
5.3	Architecture de la base de données	32
5.4	Modélisation des données métier	32
5.5	Stockage des résultats du Machine Learning	33
5.5.1	Table <code>model_runs</code>	33
5.5.2	Table <code>predictions</code>	33
5.6	Déploiement de PostgreSQL avec Docker	33
5.6.1	Configuration	33
5.6.2	Persistance des données	34
5.7	Alimentation de la base de données	34
5.8	Intégration avec Apache Spark	34
5.9	Réalisation	35
5.10	Conclusion	35
6	Visualisation des Données et Interfaces Web	36
6.1	Introduction	37
6.2	Objectif de la phase	37
6.3	Architecture de la couche de présentation	37
6.4	Interface Administrateur	38
6.5	Interface Utilisateur	39
6.6	Types de visualisations	40
6.7	Apport de la visualisation	40
6.8	Indicateurs clés de performance (KPI)	41
6.8.1	KPI liés aux modèles de machine learning	41
6.8.2	KPI liés aux données immobilières	43
6.8.3	Apport des KPI	44
6.9	Réalisation	44
6.10	Conclusion	44
	Conclusion Générale	45

Table des figures

1.1	Diagramme de cas d'utilisation global du système	4
1.2	Pipeline de traitement des données	5
1.3	Pipeline de traitement des données	6
1.4	Diagramme de classes représentant les entités de données après transformation	8
1.5	Diagramme de séquence du cas d'utilisation " Lancer la mise à jour analytique"	9
1.6	Diagramme de séquence du fonctionnement interne du pipeline	10
2.1	Composante 1 : Ingestion et nettoyage du dataset	12
2.2	Architecture Docker de la phase d'ingestion	14
2.3	Illustration du mécanisme de bind mount entre la machine hôte et le conteneur	16
2.4	Logique de traitement du script Python	17
2.5	Exécution du conteneur de prétraitement	18
3.1	Architecture du système HDFS	21
3.2	Interface web du NameNode affichant les données stockées	23
3.3	Interface web du NameNode affichant les données stockées	24
4.1	Pipeline de traitement Spark	27
4.2	Exécution du pipeline Spark	30
5.1	Tables PostgreSQL contenant les données et les résultats	35
6.1	Architecture de la couche de visualisation	38
6.2	Dashboard administrateur - performances des modèles	39
6.3	Interface utilisateur - exploration des données	40
6.4	KPI de performance des modèles de machine learning	42
6.5	KPI relatifs aux données immobilières sur app mobile	43
6.6	Vue globale de la plateforme	44

Liste des tableaux

- 1.1 Tableau des besoins fonctionnels et non fonctionnels du système 3
- 1.2 Description générale du dataset Ames Housing 7

Introduction générale

Ces dernières années, l'explosion exponentielle des données numériques a profondément transformé les systèmes informatiques modernes, donnant naissance au paradigme du *Big Data*. Ce dernier se caractérise par des volumes massifs de données, une grande variété de formats, ainsi qu'une vitesse de génération élevée, rendant les approches traditionnelles de traitement et d'analyse rapidement limitées. Face à ces défis, de nouvelles architectures distribuées et des technologies spécialisées ont émergé afin de permettre le stockage, le traitement et l'exploitation efficace de ces données à grande échelle.

Dans ce contexte, les écosystèmes Big Data tels que Hadoop et Apache Spark jouent un rôle central en permettant le traitement parallèle et distribué des données sur des clusters de machines. Ces technologies ont également favorisé l'émergence de pipelines de données automatisés, capables de transformer des données brutes en informations exploitables à travers une chaîne de traitement structurée incluant l'ingestion, le nettoyage, la transformation, l'analyse et l'apprentissage automatique.

Le présent projet s'inscrit dans cette dynamique et s'intéresse principalement à la conception et à l'implémentation d'un pipeline de traitement de données dédié au domaine immobilier, basé sur le jeu de données *Ames Housing*. L'objectif n'est pas de développer une plateforme applicative complète, mais plutôt de concevoir une architecture backend Big Data robuste, reproductible et distribuée, capable de traiter efficacement des données massives et d'en extraire des modèles prédictifs pertinents.

Pour atteindre cet objectif, une architecture conteneurisée basée sur Docker a été mise en place afin d'orchestrer les différents services du système, notamment le stockage distribué via HDFS, le traitement des données avec Apache Spark, ainsi que l'entraînement de modèles de machine learning pour la prédiction des prix immobiliers. L'ensemble du pipeline a été conçu de manière modulaire afin d'assurer la scalabilité, la maintenabilité et la séparation claire des responsabilités entre les différentes étapes du traitement.

Ainsi, ce projet illustre la mise en œuvre concrète d'un pipeline Big Data complet, allant du stockage des données brutes jusqu'à la génération de prédictions exploitables, en s'appuyant sur des technologies modernes de traitement distribué et d'orchestration de services.

ANALYSE ET CONCEPTION DU SYSTÈME

Plan

1	Introduction	3
2	Analyse des besoins	3
3	Architecture globale du système	5
4	Pipeline de traitement des données	5
5	Modélisation et description des données	6
6	Fonctionnement du système	8
7	Conclusion	10

1.1 Introduction

Ce chapitre présente l'analyse et la conception globale du système Big Data développé. L'objectif est de définir l'architecture du pipeline de traitement, les besoins du système, ainsi que les modèles de données et les interactions entre les différents composants. L'ensemble repose sur une approche distribuée utilisant Docker, HDFS et Apache Spark pour le traitement des données immobilières.

1.2 Analyse des besoins

Le système doit permettre :

Type de besoin	Description
Besoins fonctionnels	— Ingestion de données immobilières volumineuses. — Prétraitement , nettoyage et feature engineering avec Python. — Stockage distribué des données propres dans HDFS. — Entraînement distribué de modèles de machine learning avec Spark pour la prédiction. — Interface admin et utilisateur ergonomique.
Besoins non fonctionnels	— Scalabilité du système grâce à une architecture distribuée. — Reproductibilité grâce à la conteneurisation Docker. — Fiabilité et robustesse du pipeline de traitement. — Maintenabilité de l'architecture modulaire.

TABLEAU 1.1 : Tableau des besoins fonctionnels et non fonctionnels du système

1.2.1 Diagramme de cas d'utilisation

Le diagramme suivant illustre les interactions globales entre l'utilisateur et le système.

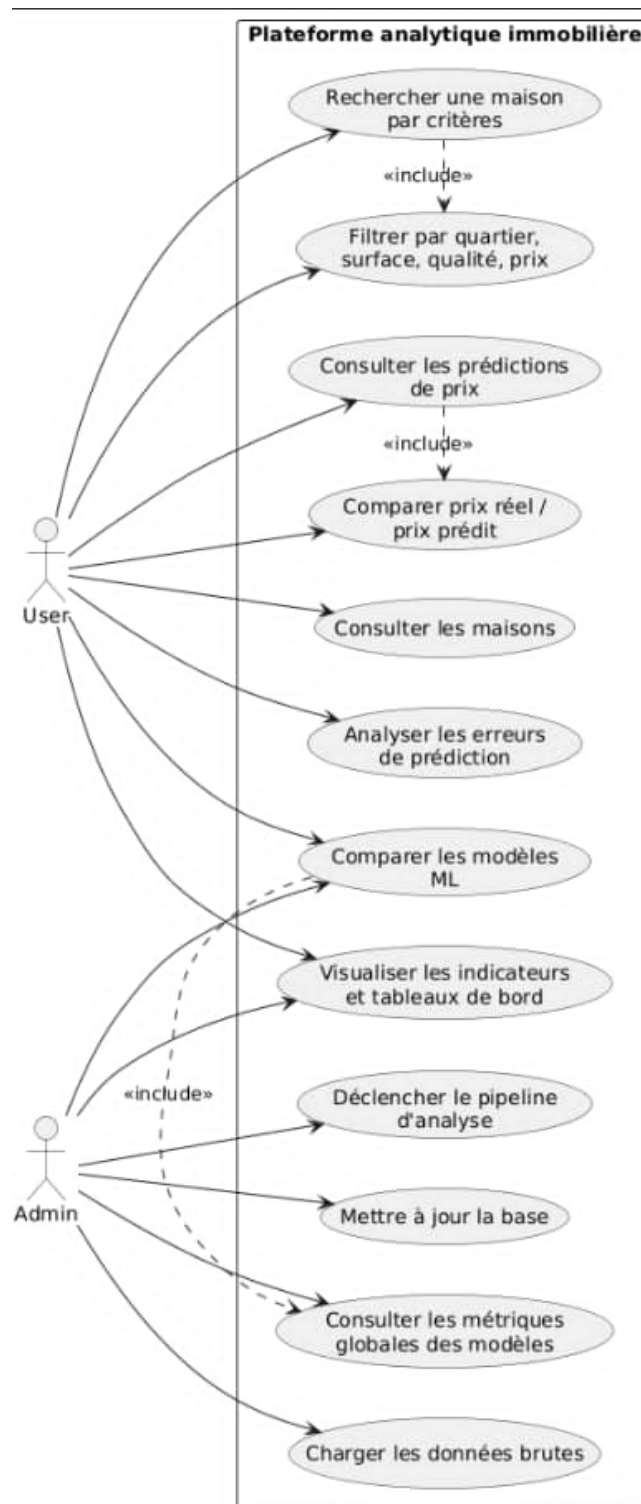


FIGURE 1.1 : Diagramme de cas d'utilisation global du système

1.3 Architecture globale du système

Le système repose sur une architecture pipeline distribuée composée de plusieurs couches comme le montre la figure 1.2 :

- Couche de stockage : HDFS
- Couche de traitement : Apache Spark
- Couche de conteneurisation : Docker
- Couche de machine learning : Spark MLlib
- Couche de présentation : Python



FIGURE 1.2 : Pipeline de traitement des données

1.4 Pipeline de traitement des données

Le pipeline de données suit une chaîne de traitement structurée comme le montre la figure ?? :

1. Ingestion des données brutes.
2. Stockage dans HDFS.
3. Prétraitement avec Spark.
4. Feature engineering.
5. Entraînement des modèles ML.
6. Évaluation et sélection du modèle.
7. Prédiction des prix.

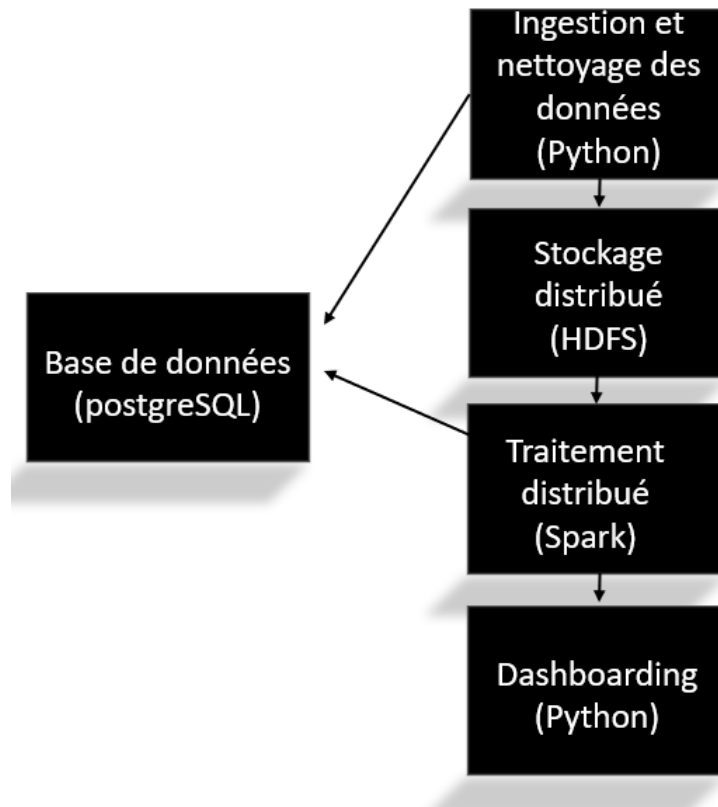


FIGURE 1.3 : Pipeline de traitement des données

1.5 Modélisation et description des données

Le système repose sur le dataset *Ames Housing*, une base de données largement utilisée dans les problématiques de prédiction des prix immobiliers. Ce dataset regroupe un ensemble riche de caractéristiques décrivant les propriétés résidentielles, notamment leurs aspects physiques, géographiques et structurels.

Dans le cadre de ce projet, ce jeu de données constitue la source principale alimentant le pipeline de traitement. Il est utilisé à la fois comme base d'apprentissage pour les modèles de machine learning, comme support pour les analyses statistiques, et comme source d'extraction de données métier pertinentes.

1.5.1 Présentation du dataset

Le tableau suivant résume les principales caractéristiques du dataset utilisé :

Caractéristique	Description
Nom du dataset	Ames Housing Dataset
Domaine	Immobilier (vente de maisons résidentielles)
Nombre de lignes	1460 observations (approximatif selon version utilisée)
Nombre de colonnes	Environ 80 variables descriptives
Type de données	Données structurées (CSV)
Types de variables	Numériques et catégorielles
Objectif principal	Prédiction du prix de vente des maisons

TABLEAU 1.2 : Description générale du dataset Ames Housing

1.5.2 Rôle du dataset dans le système

Dans l'architecture globale du système, le dataset Ames Housing constitue la source principale de données alimentant l'ensemble du pipeline de traitement. Il intervient à plusieurs niveaux du système de manière cohérente et intégrée.

- **Source du pipeline** : le dataset représente les données brutes initiales, stockées dans HDFS et utilisées comme point d'entrée du pipeline de traitement.
- **Apprentissage et prédiction** : après transformation, les données sont utilisées pour entraîner les modèles de machine learning, qui permettent ensuite de générer des prédictions de prix immobiliers.
- **Alimentation des structures de données** : les données traitées, ainsi que les résultats issus de l'apprentissage (features transformées, variables sélectionnées, modèles entraînés), sont utilisées pour alimenter des structures de données organisées facilitant l'exploitation et la prédiction.
- **Support d'analyse métier** : le dataset permet également d'extraire des informations pertinentes sur le marché immobilier, telles que les relations entre caractéristiques des biens et leurs prix.

Le diagramme de classes UML 1.4 représente les principales structures de données utilisées dans le système, notamment les variables explicatives sélectionnées ainsi que les entités intervenant dans le processus de prédiction.

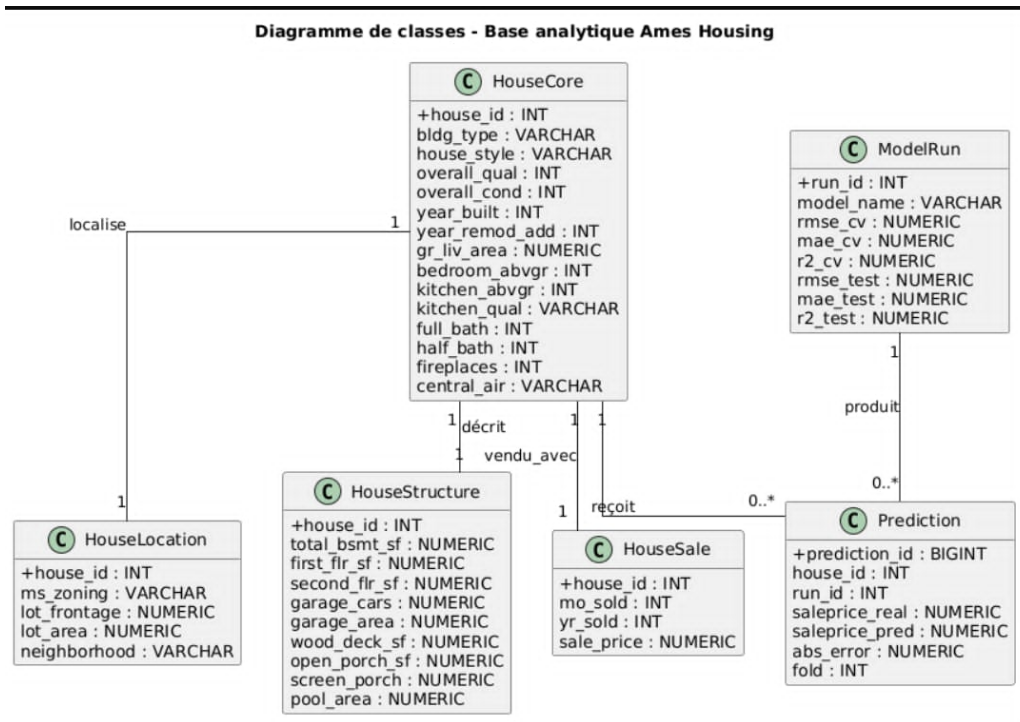


FIGURE 1.4 : Diagramme de classes représentant les entités de données après transformation

1.6 Fonctionnement du système

Cette section décrit le fonctionnement dynamique du système à travers deux niveaux de représentation : une vue externe illustrant l'interaction avec l'utilisateur, et une vue interne détaillant le traitement du pipeline de données.

1.6.1 Interaction utilisateur - système

Bien que le système soit principalement orienté backend, une interface utilisateur simplifiée permet d'interagir avec le pipeline, notamment pour lancer les traitements et consulter les résultats.

Le diagramme de séquence suivant illustre le scénario d'interaction entre l'utilisateur et le système.

Dans ce scénario, l'utilisateur déclenche une requête (exécution du pipeline ou prédiction), qui est transmise au système. Ce dernier traite la demande et retourne les résultats correspondants.

1.6.2 Fonctionnement du système

Le fonctionnement interne du système repose sur un pipeline de traitement distribué impliquant plusieurs composants tels que HDFS et Apache Spark.

Le diagramme de séquence présenté par la figure 1.5 illustre les étapes d'exécution du cas d'utilisation d'admin "Lancer la mise à jour analytique".

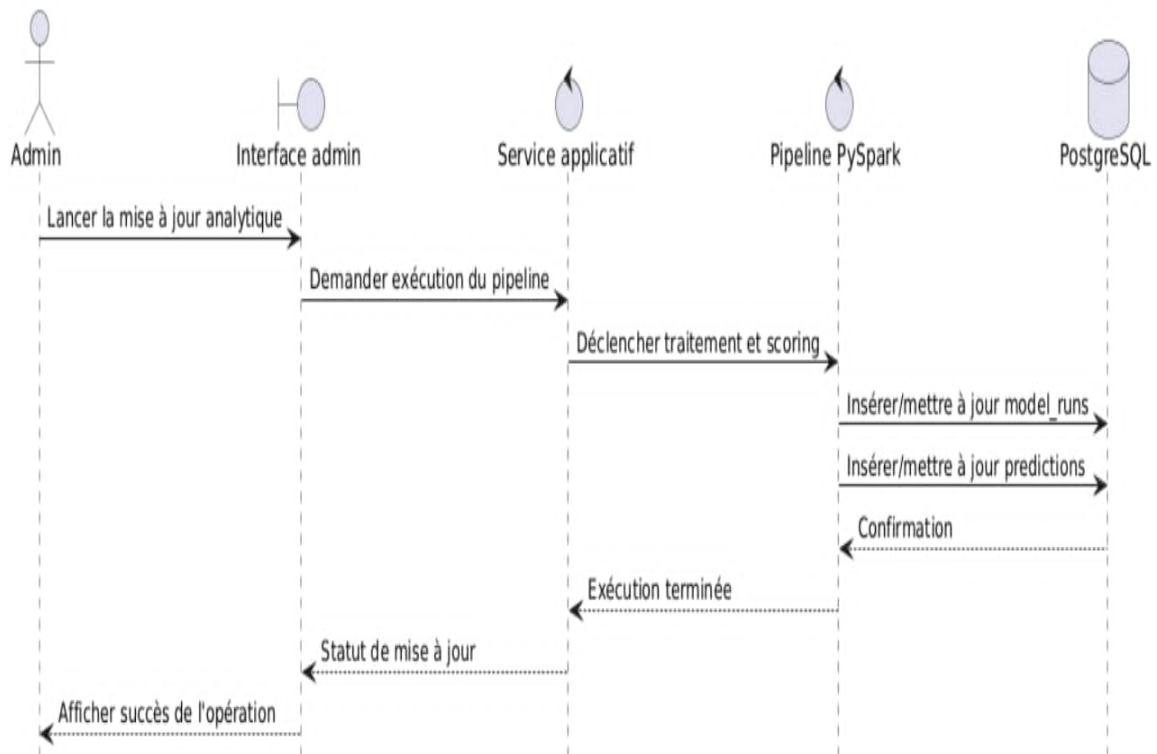


FIGURE 1.5 : Diagramme de séquence du cas d'utilisation " Lancer la mise à jour analytique"

Les actions possibles par l'utilisateur dont certains sont listé au sein du diagramme de séquence correspondant à la figure 1.3 passent par un processus impliquant différentes parties du système :

- Chargement des données dans HDFS
- Lecture des données par Spark
- Prétraitement et transformation
- Entraînement du modèle
- Génération des prédictions

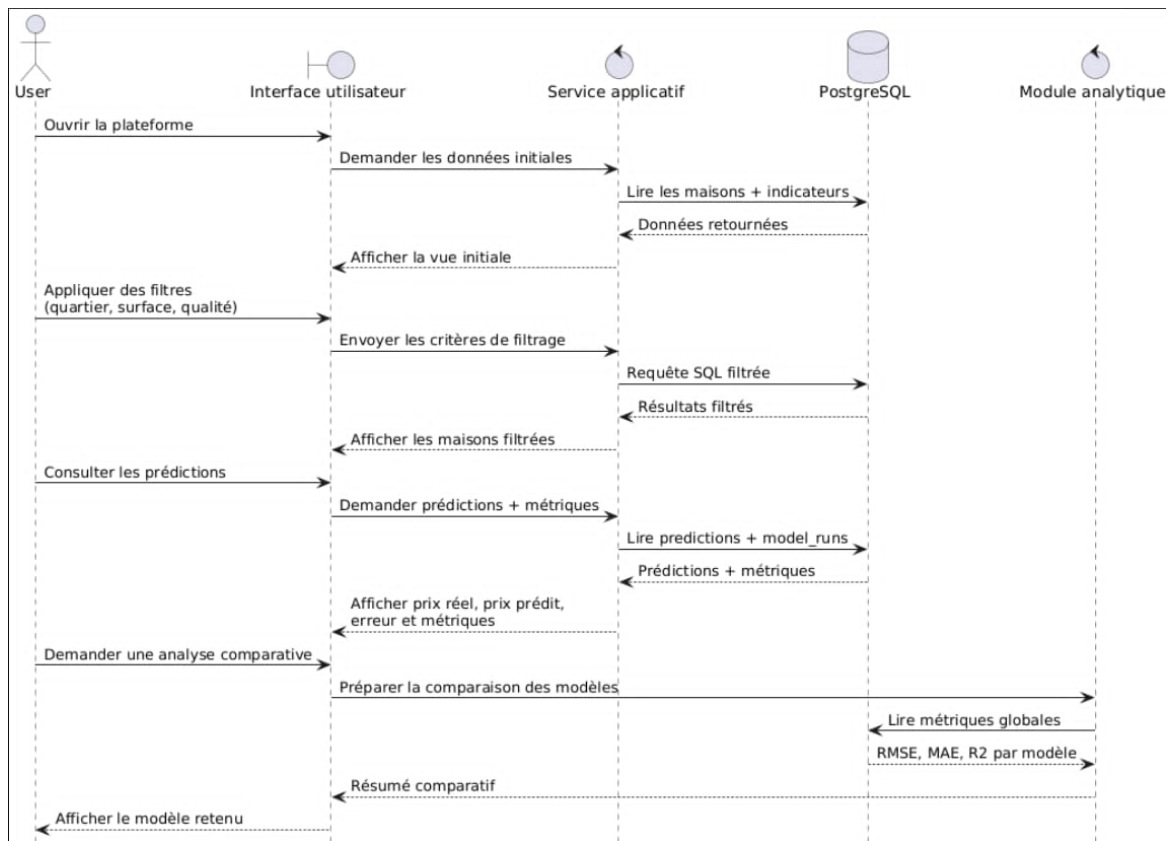


FIGURE 1.6 : Diagramme de séquence du fonctionnement interne du pipeline

1.6.3 Synthèse

Ces deux diagrammes permettent de représenter à la fois la vision externe du système (interaction utilisateur) et la logique interne du pipeline de traitement. Cette double représentation facilite la compréhension globale du fonctionnement du système.

1.7 Conclusion

Ce chapitre a permis de définir les besoins du système, son architecture globale, ainsi que la conception du pipeline de traitement des données. L'approche adoptée met en avant une architecture Big Data distribuée et conteneurisée, assurant la transformation efficace des données brutes en prédictions exploitables.

IMPLÉMENTATION DE LA PHASE D'INGESTION ET DE PRÉTRAITEMENT DES DONNÉES

Plan

1	Introduction	12
2	Objectif de la phase	12
3	Entrées et sorties	12
4	Technologies utilisées	13
5	Architecture de la solution	13
6	Réalisation	17
7	Conclusion	18

2.1 Introduction

La première étape du pipeline Big Data consiste à assurer l'ingestion et la préparation des données brutes afin de les rendre exploitables pour les phases ultérieures de traitement et de modélisation. Cette étape est cruciale, car la qualité des données conditionne directement la performance des modèles de machine learning.

Dans ce chapitre, nous détaillons l'implémentation de cette phase initiale, en mettant l'accent sur le processus d'acquisition des données, leur nettoyage, ainsi que leur transformation via des techniques de *feature engineering*. L'ensemble de ces opérations est réalisé à l'aide de scripts Python exécutés dans un environnement conteneurisé basé sur Docker.

Cette phase constitue la première “boîte noire” du pipeline, transformant un dataset brut en un ensemble de données structurées et prêtes à être exploitées dans les étapes suivantes.

2.2 Objectif de la phase

L'objectif principal de cette phase est de transformer les données brutes issues du dataset Ames Housing en un ensemble de données propres, cohérentes et exploitables.

Plus précisément, cette phase vise à :

- Assurer l'acquisition des données depuis la source initiale.
- Nettoyer les données en traitant les valeurs manquantes et incohérentes.
- Transformer les données afin de les rendre compatibles avec les modèles de machine learning.
- Générer des variables pertinentes à travers des techniques de feature engineering.
- Produire un dataset final prêt à être stocké dans HDFS.

2.3 Entrées et sorties

Cette phase du pipeline peut être décrite comme dans la figure 2.1 en tant qu'une fonction de transformation de données.

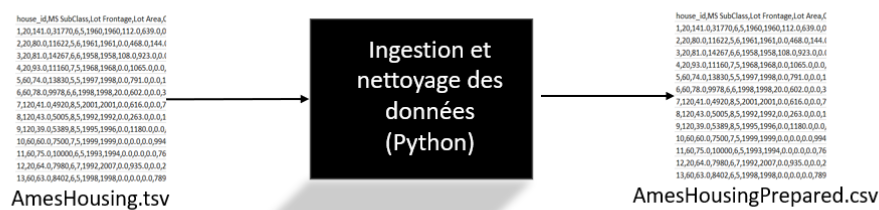


FIGURE 2.1 : Composante 1 : Ingestion et nettoyage du dataset

2.3.1 Entrée

- Dataset Ames Housing au format CSV
- Données brutes contenant des valeurs manquantes et des variables non structurées

2.3.2 Sortie

- Dataset nettoyé et transformé
- Variables normalisées et encodées
- Données prêtes pour l'entraînement des modèles
- Fichier final stocké dans HDFS

2.4 Technologies utilisées

La mise en œuvre de cette phase repose sur plusieurs technologies complémentaires :

- **Python** : utilisé pour le traitement et la transformation des données.
- **Pandas / PySpark** : manipulation et nettoyage des données.
- **Docker** : conteneurisation de l'environnement d'exécution.
- **HDFS** : stockage distribué du dataset final.

2.5 Architecture de la solution

Cette phase est comme illustré dans la figure 2.2 déployée dans un environnement conteneurisé bien configuré permettant d'assurer l'isolation et la reproductibilité des traitements.



FIGURE 2.2 : Architecture Docker de la phase d'ingestion

2.5.1 Image Docker

Une image Docker dédiée a été créée afin d'exécuter les scripts de traitement. Cette image est basé sur une image Python et contient l'ensemble des dépendances nécessaires à l'exécution du pipeline notamment les bibliothèques Numpy et Pandas.

```
FROM python:3.10-slim

WORKDIR /app

# Installer dépendances
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copier le code
COPY src/ src/

# Point d'entrée
CMD ["python", "src/prepare_ames_housing.py"]
```

2.5.2 Persistance des données avec les volumes Docker

Dans un environnement conteneurisé, les conteneurs Docker sont par nature éphémères. Cela signifie que toute donnée générée à l'intérieur d'un conteneur est susceptible d'être perdue lors de son arrêt ou de sa suppression. Cette caractéristique pose un problème majeur dans le cadre d'un pipeline de traitement de données, où la persistance des résultats est essentielle.

Afin de garantir la conservation des données traitées, une solution basée sur les volumes Docker a été mise en place. Plus précisément, des volumes de type *bind mount* ont été utilisés pour établir un lien direct entre le système de fichiers de la machine hôte et celui du conteneur.

—

2.5.2.1 Principe du bind mount

Le *bind mount* consiste à associer un répertoire local de la machine hôte à un répertoire du conteneur. Ainsi, toute modification effectuée dans ce répertoire depuis le conteneur est immédiatement répercutée sur le système hôte, et inversement.

Ce mécanisme permet de :

- Conserver les données même après la suppression du conteneur.
- Faciliter l'accès aux fichiers générés par les scripts de traitement.
- Permettre le partage de données entre plusieurs conteneurs.

—

2.5.2.2 Implémentation dans le projet

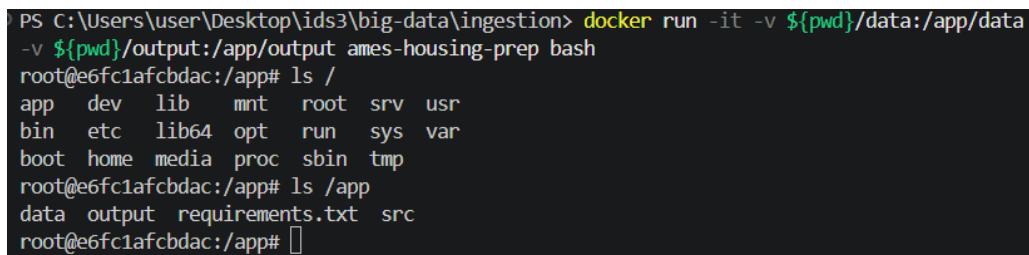
Dans cette phase du projet, deux répertoires locaux notamment `data` et `output` contenant respectivement les données d'entrée et de sortie est monté dans le conteneur de traitement ce qui est capturé dans la figure ?? . Cela permet d'assurer la persistance du dataset nettoyé et transformé.

```
docker run -it -v ${pwd}/data:/app/data -v ${pwd}/output:/app/output ames-housing-prep bash
```

Dans cet exemple, le dossier `$pwd/data` de la machine hôte est lié au répertoire `/app/data` à l'intérieur du conteneur.

2.5.2.3 Rôle dans le pipeline

L'utilisation des bind mounts permet d'assurer la continuité du pipeline de traitement en garantissant que les données transformées restent disponibles pour les étapes suivantes, notamment leur stockage dans HDFS et leur exploitation par les modules de machine learning. La capture dans la figure 2.3 montre l'existence des dossiers mappés dans le système de fichiers du conteneur.



```
PS C:\Users\user\Desktop\ids3\big-data\ingestion> docker run -it -v ${pwd}/data:/app/data
-v ${pwd}/output:/app/output ames-housing-prep bash
root@e6fc1afcbdac:/app# ls /
app  dev  lib  mnt  root  srv  usr
bin  etc  lib64  opt  run  sys  var
boot home media proc sbin tmp
root@e6fc1afcbdac:/app# ls /app
data output requirements.txt src
root@e6fc1afcbdac:/app#
```

FIGURE 2.3 : Illustration du mécanisme de bind mount entre la machine hôte et le conteneur

2.5.3 Organisation des fichiers

Le projet est structuré de manière modulaire :

- `/data` : dataset brut
- `/scripts` : scripts Python
- `/output` : données transformées
- `Dockerfile` : configuration du conteneur

2.5.4 Logique de traitement

Le script Python avec aperçue dans la figure 2.4 implémente les étapes suivantes :

- Chargement du dataset

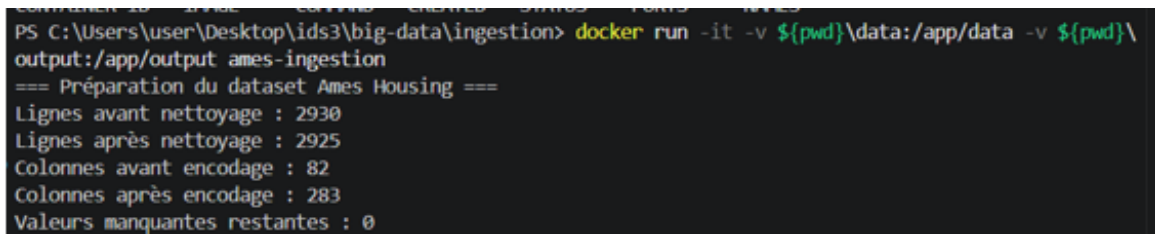
- Traitement des valeurs manquantes
- Encodage des variables catégorielles
- Normalisation des variables numériques
- Sauvegarde du dataset final

```
318 def main() -> int:
319     raw_path = data_dir() / RAW_FILENAME
320     fallback_raw_path = project_root() / RAW_FILENAME
321     prepared_path = output_dir() / PREPARED_FILENAME
322     if not raw_path.exists() and fallback_raw_path.exists():
323         raw_path = fallback_raw_path
324
325     try:
326         raw_dataframe = load_data(raw_path)
327         rows_before, columns_before = raw_dataframe.shape
328
329         cleaned_dataframe = clean_data(raw_dataframe)
330         prepared_base_dataframe = engineer_features(cleaned_dataframe)
331         prepared_base_dataframe = drop_unused_identifiers(prepared_base_dataframe)
332         prepared_base_dataframe = prepared_base_dataframe.reset_index(drop=True)
333         prepared_base_dataframe["house_id"] = prepared_base_dataframe.index + 1
334
335         prepared_dataframe, _, _ = encode_features(prepared_base_dataframe)
336         saved_path = save_prepared_data(prepared_dataframe, prepared_path)
337
338         print_summary(
339             rows_before=rows_before,
340             rows_after=len(prepared_base_dataframe),
341             columns_before=columns_before,
342             columns_after=prepared_dataframe.shape[1],
343             remaining_missing_values=int(prepared_dataframe.isna().sum().sum()),
344             output_path=saved_path,
345         )
346     except (FileNotFoundError, KeyError, ValueError) as error:
347         print(f"Erreur: {error}", file=sys.stderr)
348         return 1
349
350 if __name__ == "__main__":
351     raise SystemExit(main())
```

FIGURE 2.4 : Logique de traitement du script Python

2.6 Réalisation

Cette phase a été testée avec succès dans l'environnement Docker. Le pipeline d'ingestion et de prétraitement permet de transformer efficacement les données brutes en données exploitables et ce est prouvé par la figure 2.5.



```
PS C:\Users\user\Desktop\ids3\big-data\ingestion> docker run -it -v ${pwd}\data:/app/data -v ${pwd}\
output:/app/output ames-ingestion
=== Préparation du dataset Ames Housing ===
Lignes avant nettoyage : 2930
Lignes après nettoyage : 2925
Colonnes avant encodage : 82
Colonnes après encodage : 283
Valeurs manquantes restantes : 0
```

FIGURE 2.5 : Exécution du conteneur de prétraitement

2.7 Conclusion

Cette phase constitue une étape fondamentale du pipeline Big Data. Elle permet de garantir la qualité des données avant leur exploitation dans les modèles de machine learning. Grâce à l'utilisation de Docker, cette phase est reproductible, modulaire et facilement intégrable dans une architecture distribuée.

IMPLÉMENTATION DE LA PHASE DE STOCKAGE DISTRIBUÉ DANS HDFS

Plan

1	Introduction	20
2	Objectif de la phase	20
3	Entrées et sorties	20
4	Architecture de HDFS	21
5	HDFS et le Big Data	21
6	Implémentation du mini-cluster HDFS	21
7	Intégration dans le pipeline	23
8	Réalisation	23
9	Conclusion	24

3.1 Introduction

Après la phase d’ingestion et de prétraitement, les données obtenues sont propres, structurées et prêtes à être exploitées. Toutefois, afin de permettre leur traitement à grande échelle, il est nécessaire de les stocker dans un système distribué adapté aux environnements Big Data.

Dans ce contexte, Hadoop Distributed File System (HDFS) est utilisé comme solution de stockage. Ce système permet de gérer de grands volumes de données tout en assurant leur disponibilité, leur tolérance aux pannes et leur distribution sur plusieurs nœuds.

Ce chapitre présente l’implémentation de cette phase de stockage, en décrivant l’architecture de HDFS, son intégration dans le pipeline, ainsi que la mise en place d’un mini-cluster à l’aide de Docker.

3.2 Objectif de la phase

L’objectif principal de cette phase est de stocker les données prétraitées dans un système distribué afin de les rendre accessibles aux traitements parallèles ultérieurs.

Plus précisément, cette phase vise à :

- Assurer le stockage distribué des données nettoyées.
- Garantir la tolérance aux pannes et la disponibilité des données.
- Préparer les données pour leur exploitation par Apache Spark.
- Permettre une lecture parallèle des données dans les phases suivantes du pipeline.

3.3 Entrées et sorties

3.3.1 Entrée

- Données nettoyées et transformées issues de la phase précédente
- Fichiers au format structuré (CSV ou Parquet)

3.3.2 Sortie

- Données stockées dans HDFS
- Fichiers accessibles de manière distribuée
- Dataset prêt pour l’entraînement avec Spark

3.4 Architecture de HDFS

HDFS repose sur une architecture distribuée de type maître-esclave comme montré dans la figure 3.1, composée principalement de deux types de nœuds :

- **NameNode** : responsable de la gestion des métadonnées (structure des fichiers, localisation des blocs).
- **DataNodes** : responsables du stockage effectif des blocs de données.

Les fichiers sont divisés en blocs et distribués sur plusieurs DataNodes, ce qui permet une lecture parallèle et une meilleure performance dans les traitements Big Data.

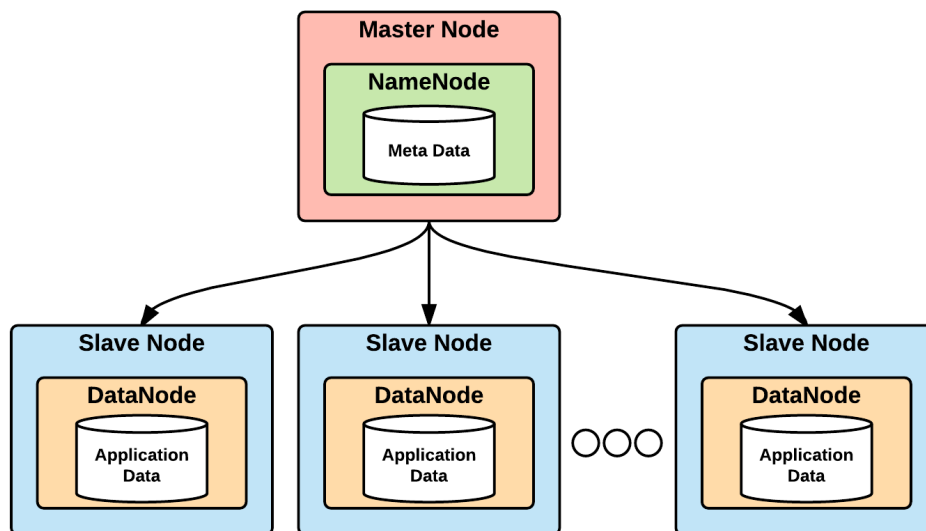


FIGURE 3.1 : Architecture du système HDFS

3.5 HDFS et le Big Data

HDFS est particulièrement adapté aux environnements Big Data grâce à ses caractéristiques :

- **Scalabilité** : possibilité d'ajouter facilement de nouveaux nœuds.
- **Tolérance aux pannes** : réplication des blocs de données.
- **Traitement parallèle** : lecture simultanée des données.
- **Optimisation pour gros volumes** : gestion efficace des fichiers volumineux.

3.6 Implémentation du mini-cluster HDFS

Afin de simuler un environnement distribué, un mini-cluster de conteneurs HDFS a été déployé à l'aide de Docker Compose.

3.6.1 Architecture du cluster

Le cluster est composé de :

- Un conteneur NameNode
- Un ou plusieurs conteneurs DataNode

3.6.2 Configuration Docker Compose

```
version: "3.8"

services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    environment:
      - CLUSTER_NAME=test-cluster
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    ports:
      - "9870:9870"
      - "9000:9000"
    volumes:
      - namenode_data:/hadoop/dfs/name
      - ../ingestion/output:/data-input

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    depends_on:
      - namenode
    ports:
      - "9864:9864"
    volumes:
      - datanode_data:/hadoop/dfs/data
```

volumes :

namenode_data:

datanode_data:

3.6.3 Mapping des ports

Le mapping des ports permet d'accéder aux interfaces web des composants HDFS :

- 9870 : interface web du NameNode
- 9864 : interface web du DataNode

3.7 Intégration dans le pipeline

Après leur transformation, les données sont transférées vers HDFS afin d'être utilisées par Apache Spark dans la phase suivante.

Ce stockage constitue une étape clé permettant de découpler les phases de traitement et d'assurer une meilleure performance globale du système. La figure 3.2 montre l'état résultant de ce composant dans le pipeline

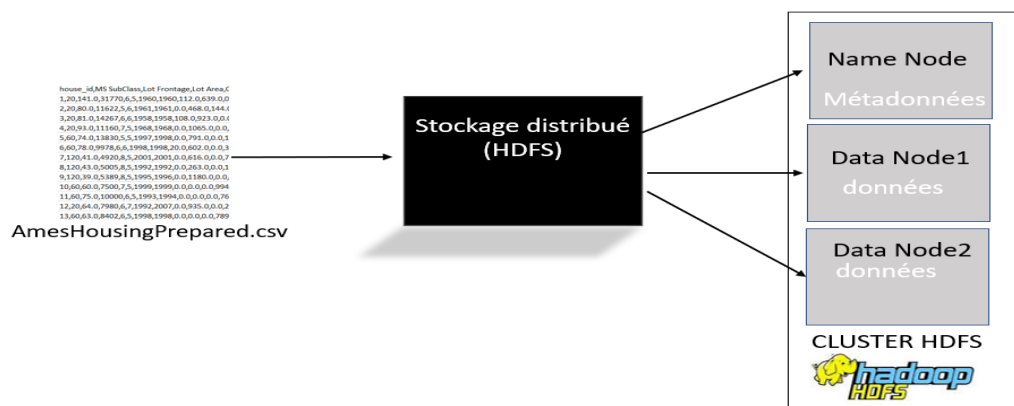


FIGURE 3.2 : Interface web du NameNode affichant les données stockées

3.8 Réalisation

La mise en place du mini-cluster HDFS a été validée avec succès. Les données prétraitées ont été correctement stockées et sont accessibles via l'interface web du NameNode comme dans la figure 3.3.

Configured Capacity:	1006.85 GB
Configured Remote Capacity:	0 B
DFS Used:	1.86 MB (0%)
Non DFS Used:	5.5 GB
DFS Remaining:	950.14 GB (94.37%)
Block Pool Used:	1.86 MB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	2
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Wed Apr 22 01:08:46 +0200 2026
Last Checkpoint Time	Wed Apr 22 01:08:46 +0200 2026
Enabled Erasure Coding Policies	RS-6-3-1024k

FIGURE 3.3 : Interface web du NameNode affichant les données stockées

3.9 Conclusion

Cette phase a permis de mettre en place une solution de stockage distribuée adaptée aux besoins du projet. L'utilisation de HDFS garantit la disponibilité, la scalabilité et la performance des données, tout en préparant efficacement leur exploitation dans la phase de traitement avec Apache Spark.

IMPLÉMENTATION DE LA PHASE DE TRAITEMENT ET DE MODÉLISATION AVEC APACHE SPARK

Plan

1	Introduction	26
2	Objectif de la phase	26
3	Entrées et sorties	26
4	Pipeline de traitement	27
5	Préparation des données	27
6	Modèles de machine learning	28
7	Évaluation des performances	28
8	Stockage des résultats	28
9	Réalisation	29
10	Conclusion	30

4.1 Introduction

Après le stockage des données dans HDFS, la phase suivante du pipeline consiste à exploiter ces données à travers des traitements distribués et des modèles de machine learning. Cette étape représente le cœur du système, car elle permet de transformer les données en connaissances exploitables.

Dans ce chapitre, nous détaillons l'implémentation de la phase de traitement et de modélisation à l'aide d'Apache Spark. Cette phase comprend la lecture des données depuis HDFS, la préparation des variables, l'entraînement de plusieurs modèles de régression, ainsi que l'évaluation et le stockage des résultats dans une base de données PostgreSQL.

L'objectif est de construire un pipeline complet capable non seulement de prédire les prix immobiliers, mais également de conserver les performances des modèles pour une exploitation future.

4.2 Objectif de la phase

Cette phase vise à exploiter les données stockées dans HDFS afin de produire des prédictions fiables et mesurables.

Les objectifs principaux sont :

- Charger les données depuis HDFS dans Spark
- Construire un vecteur de features exploitable
- Entraîner plusieurs modèles de machine learning
- Évaluer les performances des modèles
- Stocker les résultats et les prédictions dans une base de données

4.3 Entrées et sorties

4.3.1 Entrée

- Dataset nettoyé stocké dans HDFS

4.3.2 Sortie

- Modèles entraînés
- Prédictions sur les données de test
- Table `model_runs` contenant les métriques
- Table `predictions` contenant les résultats détaillés

4.4 Pipeline de traitement

Le pipeline Spark suit les étapes suivantes :

1. Lecture des données depuis HDFS
2. Préparation des colonnes (sanitization)
3. Sélection des variables numériques
4. Construction du vecteur de features
5. Séparation en données d'entraînement et de test
6. Entraînement des modèles
7. Évaluation des performances
8. Stockage des résultats

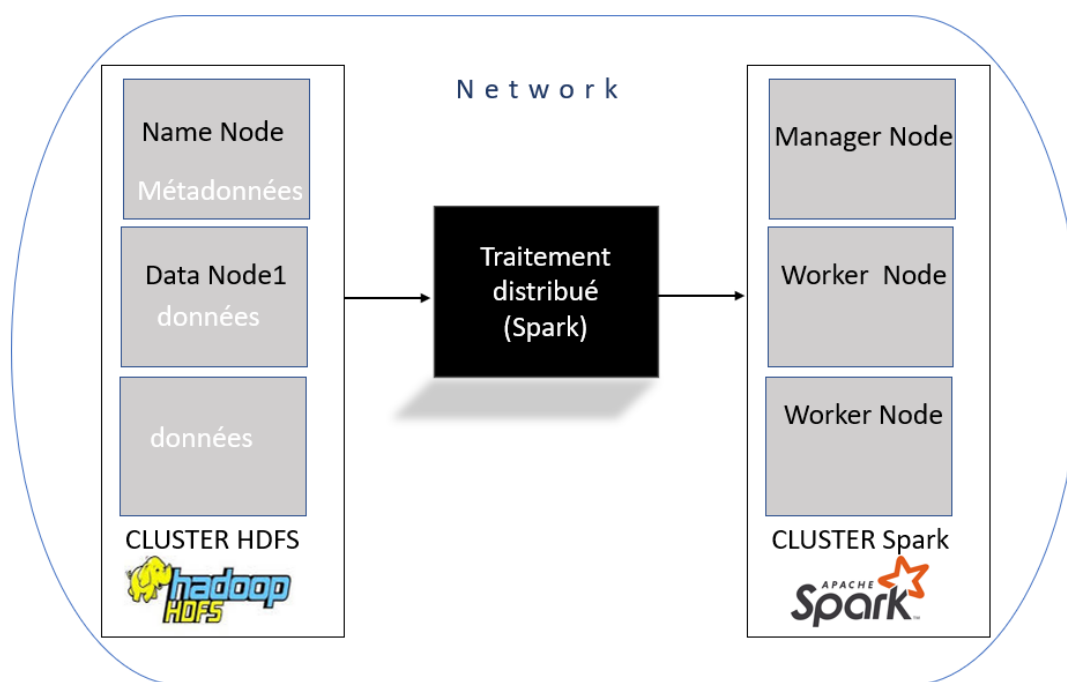


FIGURE 4.1 : Pipeline de traitement Spark

4.5 Préparation des données

Les données sont lues depuis HDFS puis transformées afin d'être compatibles avec les modèles de machine learning.

Les principales étapes sont :

- Nettoyage des noms de colonnes (sanitization)

- Suppression des valeurs nulles critiques
- Sélection des variables numériques
- Construction du vecteur de features avec `VectorAssembler`

```
assembler = VectorAssembler(  
    inputCols=feature_cols,  
    outputCol="features"  
)
```

4.6 Modèles de machine learning

Trois modèles de régression ont été implémentés afin de comparer leurs performances :

4.6.1 Régression Linéaire

Modèle simple permettant d'établir une relation linéaire entre les variables explicatives et la variable cible.

4.6.2 Random Forest

Modèle basé sur un ensemble d'arbres de décision, permettant de capturer des relations non linéaires.

4.6.3 Gradient Boosting

Modèle avancé qui améliore progressivement les prédictions en corrigeant les erreurs des modèles précédents.

4.7 Évaluation des performances

Les modèles sont évalués à l'aide des métriques suivantes :

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 (Coefficient de détermination)

4.8 Stockage des résultats

Contrairement à un pipeline classique limité à la prédiction, ce système permet également de stocker les performances des modèles et les résultats des prédictions dans une base de données PostgreSQL.

4.8.1 Table des exécutions (model_runs)

Cette table contient les métriques d'évaluation de chaque modèle :

- Nom du modèle
- RMSE, MAE, R^2 (train et test)

4.8.2 Table des prédictions (predictions)

Cette table contient :

- Identifiant du modèle
- Identifiant du bien immobilier
- Valeur réelle
- Valeur prédite
- Erreur absolue

Ce stockage permet de :

- Comparer les performances des modèles
- Conserver un historique des entraînements
- Analyser les erreurs de prédiction
- Faciliter la sélection du meilleur modèle

4.9 Réalisation

Le pipeline Spark a été exécuté avec succès dans un environnement distribué. Les modèles ont été entraînés et les résultats ont été stockés dans PostgreSQL.

 **Spark Master at**
spark://a14bb28df5c1:7077
URL: spark://a14bb28df5c1:7077
Alive Workers: 1
Cores in use: 8 Total, 0 Used
Memory in use: 6.7 GiB Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

▼ **Workers (1)**

Worker Id	Address	State	Cores	Memory	Resources
worker-20260422002343-172.18.0.4-41775	172.18.0.4:41775	ALIVE	8 (0 Used)	6.7 GiB (0.0 B Used)	

FIGURE 4.2 : Exécution du pipeline Spark

4.10 Conclusion

Cette phase représente le cœur du système, permettant de transformer les données en connaissances exploitables. L'intégration de Spark et PostgreSQL permet non seulement de générer des prédictions, mais également de structurer les résultats pour une analyse approfondie et une amélioration continue du modèle.

IMPLÉMENTATION DE LA COUCHE DE STOCKAGE ET D'EXPLOITATION DES DONNÉES AVEC POSTGRESQL

Plan

1	Introduction	32
2	Objectif de la phase	32
3	Architecture de la base de données	32
4	Modélisation des données métier	32
5	Stockage des résultats du Machine Learning	33
6	Déploiement de PostgreSQL avec Docker	33
7	Alimentation de la base de données	34
8	Intégration avec Apache Spark	34
9	Réalisation	35
10	Conclusion	35

5.1 Introduction

Après les phases de traitement et de modélisation, il est nécessaire de stocker les résultats obtenus ainsi que certaines données structurées afin de permettre leur exploitation. Cette étape constitue la couche finale du pipeline, assurant la persistance, l'analyse et la valorisation des données.

Dans ce projet, PostgreSQL est utilisé comme système de gestion de base de données relationnelle. Il permet de stocker à la fois les données métier issues du dataset ainsi que les résultats des modèles de machine learning, notamment les métriques et les prédictions.

Ce chapitre présente la conception de la base de données, son intégration dans l'architecture du système, ainsi que les mécanismes d'alimentation mis en place.

5.2 Objectif de la phase

Cette phase vise à structurer et stocker les données afin de permettre leur exploitation dans des contextes analytiques et applicatifs.

Les objectifs sont :

- Stocker les données métier sous forme normalisée
- Conserver les résultats des modèles de machine learning
- Permettre l'analyse des performances des modèles
- Faciliter l'accès aux données pour une interface utilisateur ou des outils analytiques

5.3 Architecture de la base de données

La base de données est organisée en deux parties principales :

- **Schéma métier** : représentant les informations immobilières issues du dataset
- **Schéma analytique** : contenant les résultats du machine learning

5.4 Modélisation des données métier

Afin de réduire la redondance et améliorer la structuration des données, une approche normalisée a été adoptée. Les données ont été réparties en plusieurs tables thématiques :

- `houses_core` : informations principales du bien
- `houses_location` : localisation
- `houses_structure` : caractéristiques physiques
- `houses_sale` : informations de vente

Ces tables sont liées entre elles via la clé primaire `house_id`, assurant l'intégrité référentielle.

5.5 Stockage des résultats du Machine Learning

En complément des données métier, deux tables ont été conçues pour stocker les résultats du pipeline de machine learning.

5.5.1 Table `model_runs`

Cette table enregistre les performances de chaque modèle entraîné :

- Nom du modèle
- RMSE, MAE, R^2 (train et test)

5.5.2 Table `predictions`

Cette table contient les résultats détaillés des prédictions :

- Identifiant du bien immobilier
- Valeur réelle
- Valeur prédite
- Erreur
- Référence au modèle utilisé

Cette architecture permet de :

- Suivre l'évolution des performances des modèles
- Comparer différents algorithmes
- Identifier les erreurs de prédiction
- Construire une base d'analyse pour des améliorations futures

5.6 Déploiement de PostgreSQL avec Docker

Le service PostgreSQL est déployé sous forme de conteneur Docker afin de garantir la portabilité et la reproductibilité de l'environnement.

5.6.1 Configuration

`services:`

`postgres:`


```
image: postgres:16

container_name: ames-postgres

ports:
  - "5432:5432"

volumes:
  - postgres_data:/var/lib/postgresql/data
```

5.6.2 Persistance des données

Un volume Docker est utilisé pour conserver les données de la base PostgreSQL, même après l'arrêt du conteneur. Cela garantit la durabilité des informations stockées.

5.7 Alimentation de la base de données

Un script Python a été développé pour importer et structurer les données du dataset dans la base PostgreSQL.

Les principales étapes sont :

- Lecture du dataset
- Nettoyage des valeurs
- Transformation des types
- Répartition des données dans les tables normalisées

Chaque ligne du dataset est transformée en plusieurs enregistrements répartis dans les différentes tables, selon leur nature.

5.8 Intégration avec Apache Spark

Les résultats du pipeline Spark sont directement insérés dans PostgreSQL via une connexion JDBC.

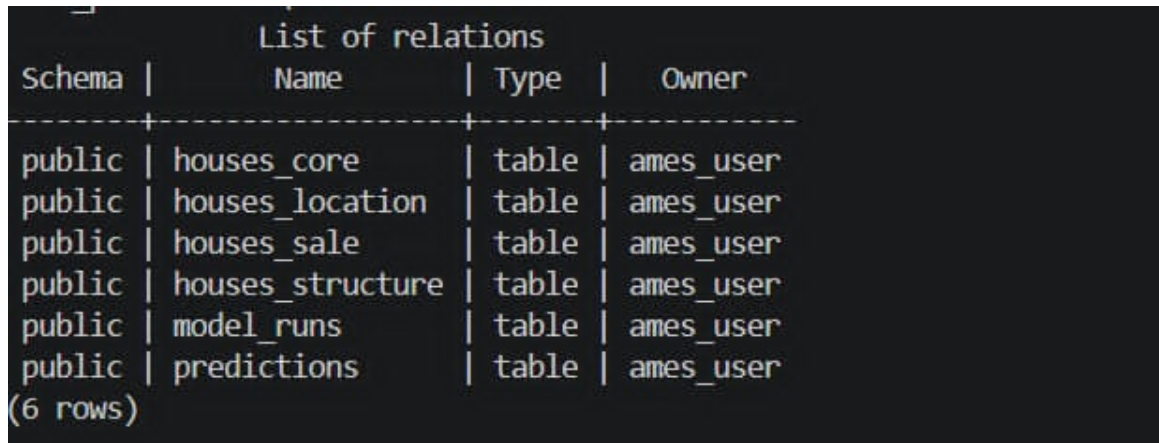
Cela permet de relier la couche Big Data (HDFS + Spark) avec une couche relationnelle exploitable.

Cette intégration assure :

- La centralisation des résultats
- La traçabilité des modèles
- Une exploitation facile des données

5.9 Réalisation

Le système a été testé avec succès. Les données métier ainsi que les résultats des modèles ont été correctement stockés dans la base PostgreSQL.



```

List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | houses_core    | table | ames_user
public | houses_location | table | ames_user
public | houses_sale    | table | ames_user
public | houses_structure | table | ames_user
public | model_runs     | table | ames_user
public | predictions    | table | ames_user
(6 rows)

```

FIGURE 5.1 : Tables PostgreSQL contenant les données et les résultats

5.10 Conclusion

Cette phase permet de compléter le pipeline en assurant la persistance et la structuration des données. L'utilisation de PostgreSQL offre une couche d'exploitation essentielle, permettant d'analyser les résultats du machine learning et de construire une base de données métier cohérente.

VISUALISATION DES DONNÉES ET INTERFACES WEB

Plan

1	Introduction	37
2	Objectif de la phase	37
3	Architecture de la couche de présentation	37
4	Interface Administrateur	38
5	Interface Utilisateur	39
6	Types de visualisations	40
7	Apport de la visualisation	40
8	Indicateurs clés de performance (KPI)	41
9	Réalisation	44
10	Conclusion	44

6.1 Introduction

Après les phases de traitement, de modélisation et de stockage, la dernière étape du pipeline consiste à valoriser les données à travers des interfaces de visualisation. Cette couche permet de rendre les résultats compréhensibles et exploitables par les utilisateurs.

Dans ce projet, une interface web a été développée afin de permettre l'accès aux données et aux résultats du machine learning sous forme de tableaux de bord interactifs. Deux types d'interfaces ont été conçus : une interface administrateur pour le suivi technique du pipeline, et une interface utilisateur orientée consultation et analyse.

6.2 Objectif de la phase

Cette phase vise à transformer les données stockées en informations visuelles exploitables.

Les objectifs principaux sont :

- Visualiser les données immobilières
- Afficher les performances des modèles
- Permettre l'exploration interactive des données
- Faciliter la prise de décision basée sur les résultats

6.3 Architecture de la couche de présentation

La couche de présentation repose sur une architecture web connectée à la base PostgreSQL.

Elle comprend :

- Interface web (frontend)
- API backend (connexion à PostgreSQL)
- Base de données (données métier + résultats ML)

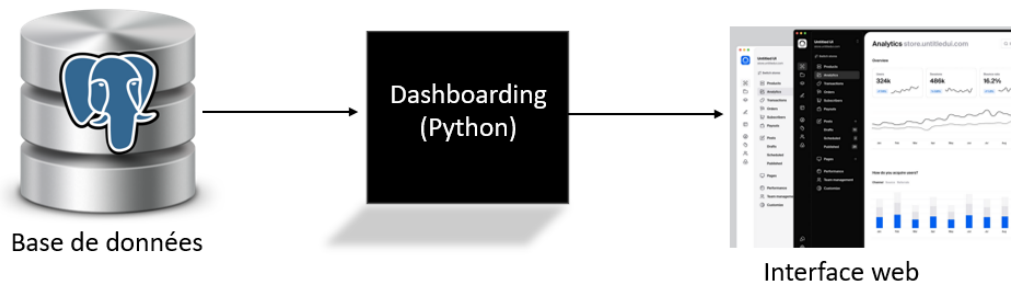


FIGURE 6.1 : Architecture de la couche de visualisation

6.4 Interface Administrateur

L'interface administrateur permet de superviser le fonctionnement du pipeline et d'analyser les performances des modèles.

Elle offre les fonctionnalités suivantes :

- Visualisation des performances des modèles (RMSE, MAE, R^2)
- Comparaison entre différents algorithmes
- Suivi des exécutions (model runs)
- Analyse des erreurs de prédiction

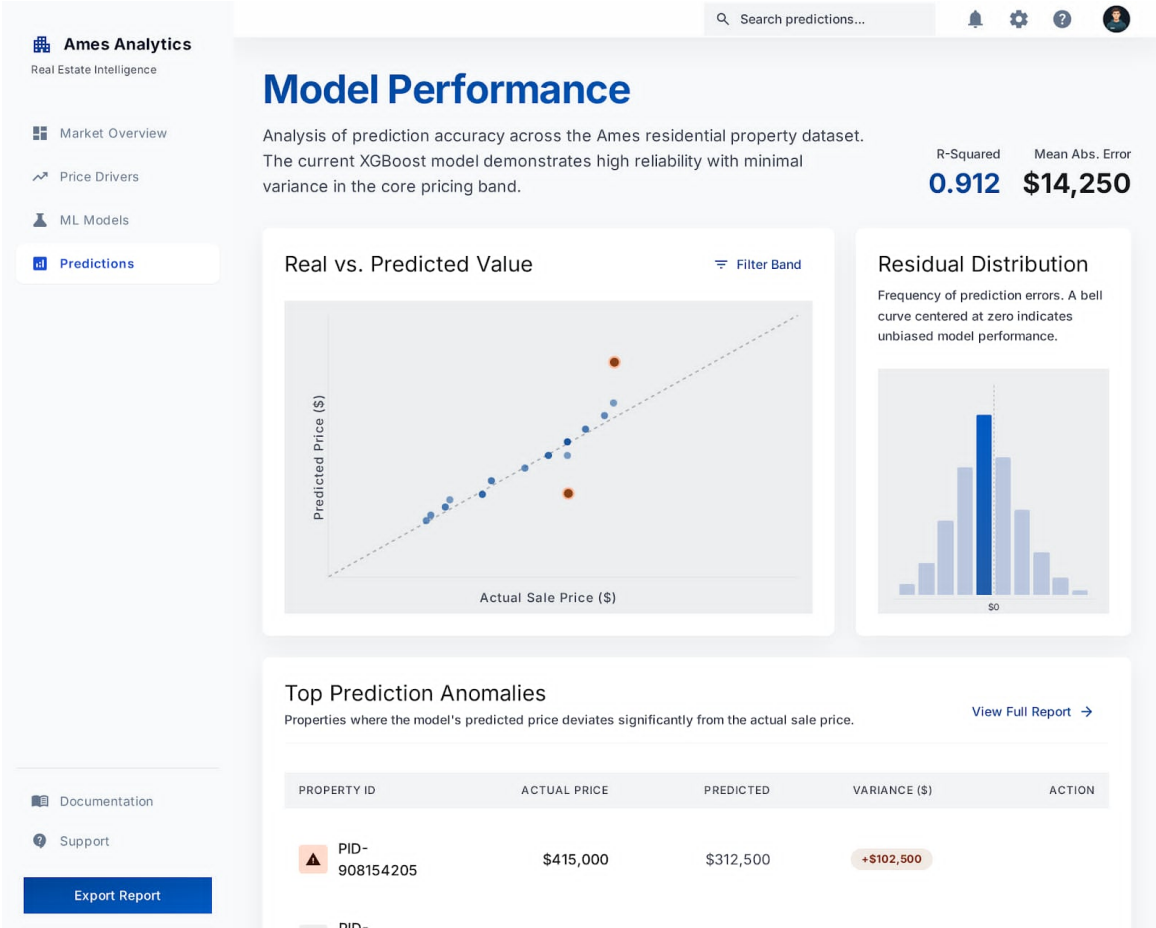


FIGURE 6.2 : Dashboard administrateur - performances des modèles

6.5 Interface Utilisateur

L'interface utilisateur est destinée à l'exploration des données immobilières et à la consultation des prédictions.

Elle permet :

- Rechercher des biens immobiliers
- Visualiser les caractéristiques d'un bien
- Consulter les prix prédits
- Explorer les tendances du marché

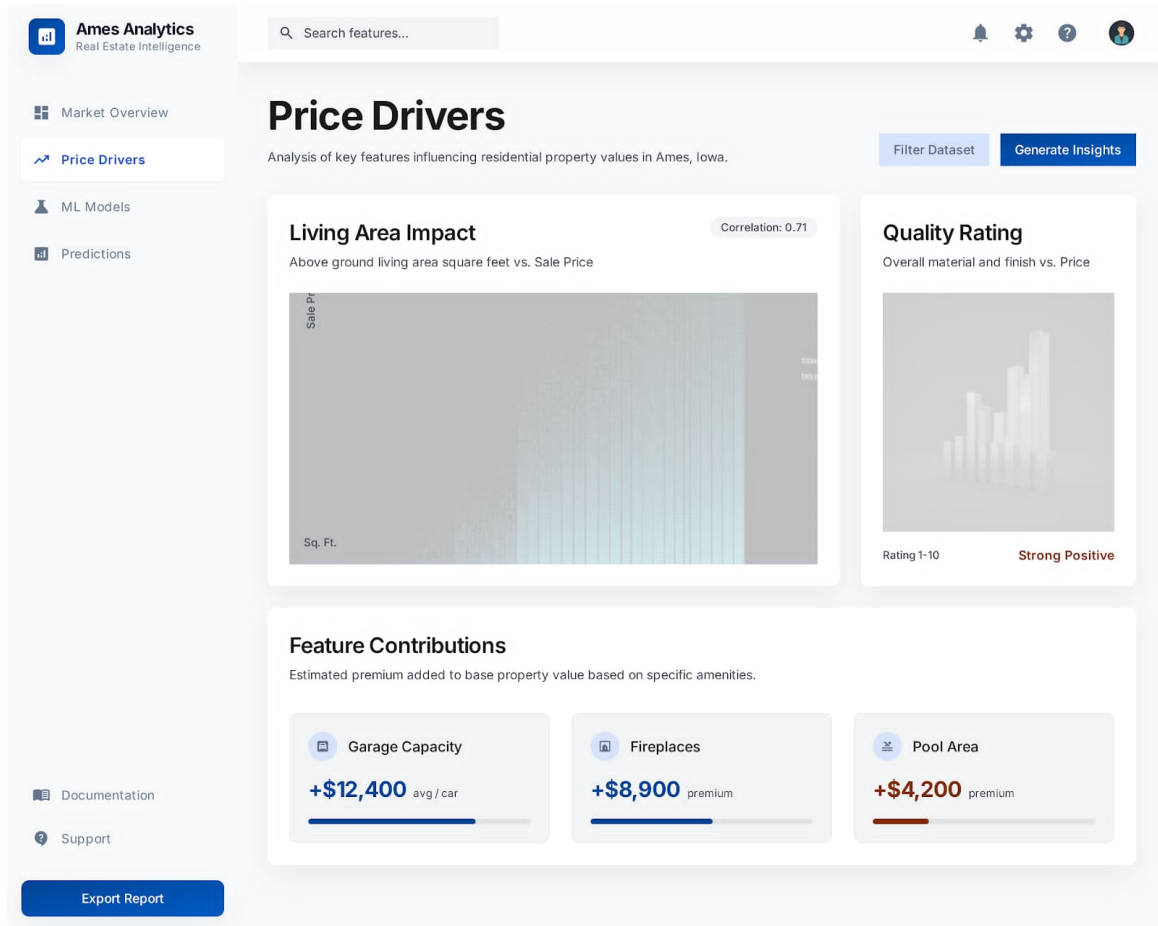


FIGURE 6.3 : Interface utilisateur - exploration des données

6.6 Types de visualisations

Plusieurs types de graphiques ont été utilisés afin de représenter efficacement les données :

- Graphiques en barres : comparaison des modèles
- Graphiques en lignes : évolution des prix
- Histogrammes : distribution des variables
- Tableaux : affichage des prédictions détaillées

6.7 Apport de la visualisation

La couche de visualisation permet de transformer les résultats techniques en informations compréhensibles.

Elle apporte :

- Une meilleure interprétation des modèles
- Une analyse rapide des performances
- Une interaction directe avec les données

- Une valorisation du pipeline Big Data

6.8 Indicateurs clés de performance (KPI)

Afin de faciliter l'analyse et la prise de décision, le système propose un ensemble d'indicateurs clés de performance (Key Performance Indicators - KPI). Ces indicateurs permettent de résumer les informations essentielles issues des données immobilières et des modèles de prédiction.

6.8.1 KPI liés aux modèles de machine learning

Ces indicateurs permettent d'évaluer la qualité des modèles entraînés :

- **RMSE (Root Mean Squared Error)** : mesure l'écart moyen entre les valeurs réelles et les valeurs prédites. Plus cette valeur est faible, plus le modèle est performant.
- **MAE (Mean Absolute Error)** : représente l'erreur moyenne absolue des prédictions, offrant une interprétation directe en unités monétaires.
- **R^2 (Coefficient de détermination)** : indique la capacité du modèle à expliquer la variance des données. Une valeur proche de 1 traduit un bon ajustement.
- **Nombre de modèles entraînés** : permet de suivre les différentes expérimentations réalisées dans le système.

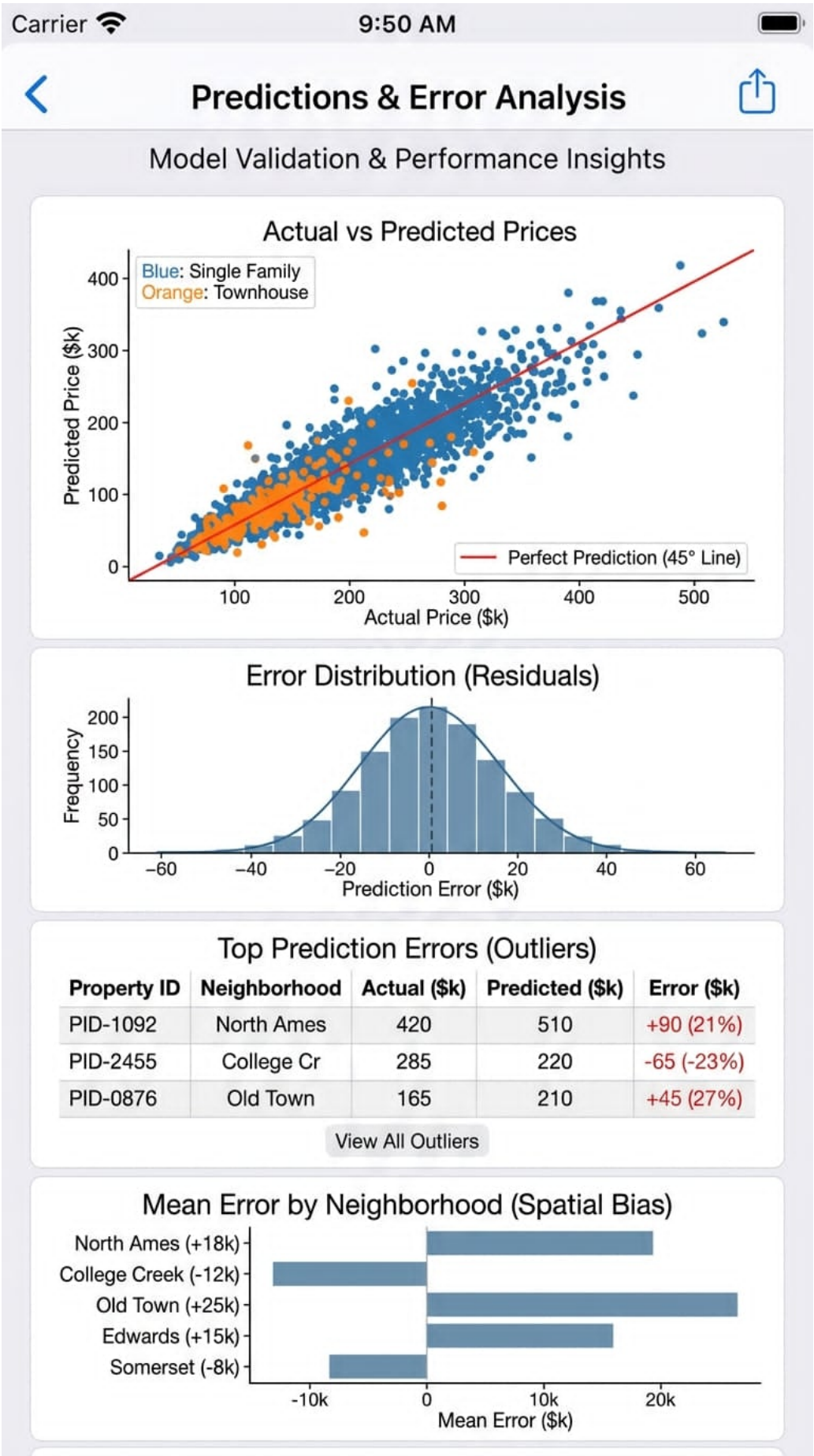


FIGURE 6.4 : KPI de performance des modèles de machine learning

6.8.2 KPI liés aux données immobilières

Ces indicateurs permettent de mieux comprendre le marché immobilier :

- **Prix moyen des maisons** : indicateur global du marché.
- **Prix maximum et minimum** : permet d'identifier les extrêmes du marché.
- **Surface moyenne habitable** : donne une idée des caractéristiques des biens.
- **Nombre total de biens analysés** : reflète le volume de données exploitées.

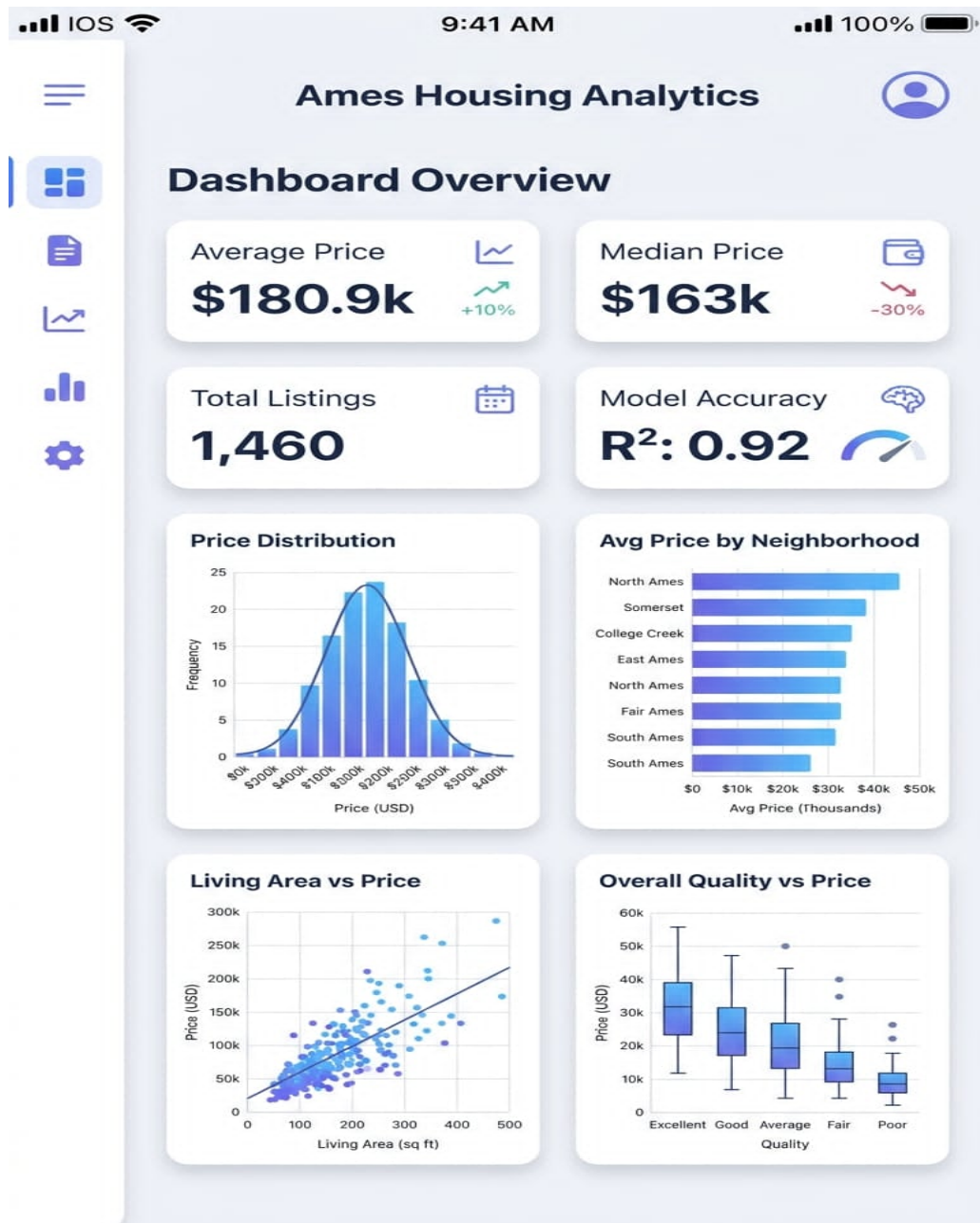


FIGURE 6.5 : KPI relatifs aux données immobilières sur app mobile

6.8.3 Apport des KPI

Les KPI jouent un rôle essentiel dans le système en permettant :

- Une évaluation rapide des performances des modèles
- Une compréhension synthétique des données
- Une aide à la décision pour les utilisateurs
- Une surveillance continue du système

6.9 Réalisation

Les interfaces ont été développées et testées avec succès. Les dashboards affichent correctement les données issues de PostgreSQL, ainsi que les résultats des modèles de machine learning.

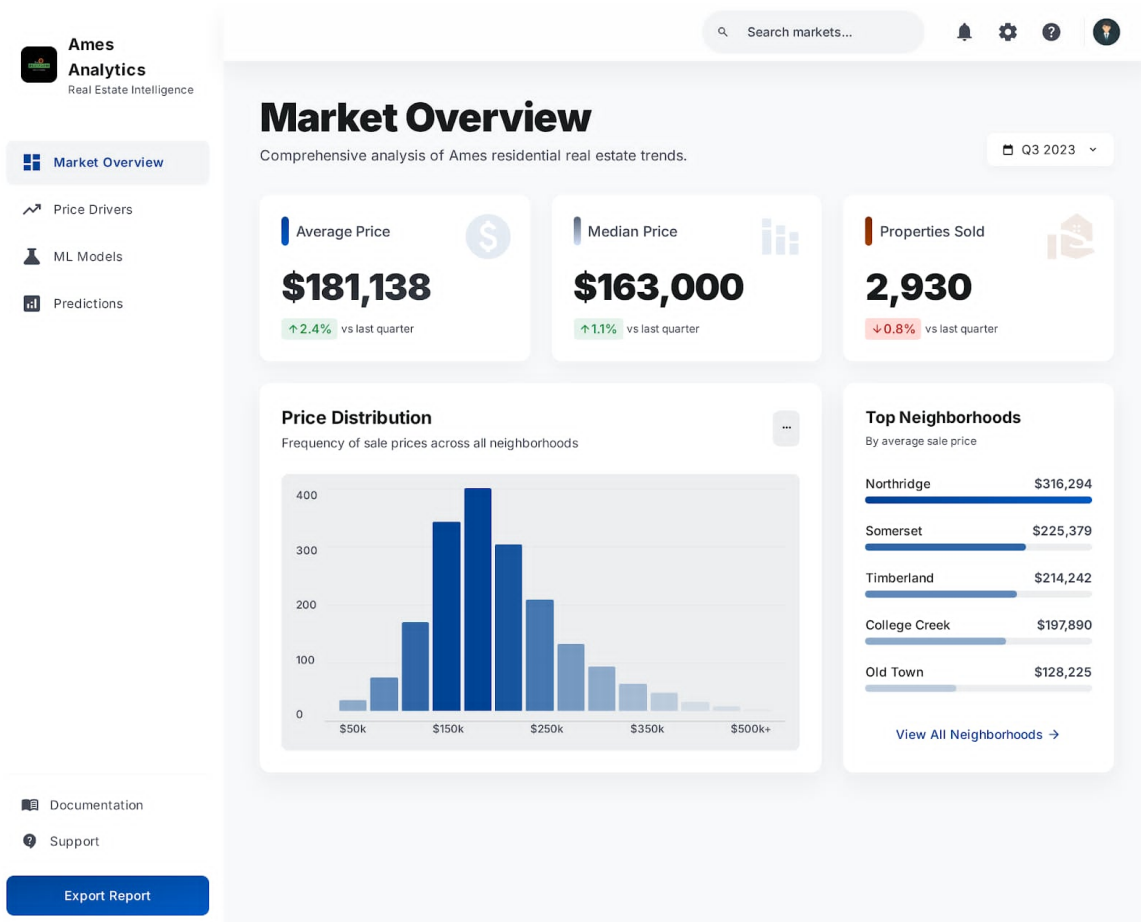


FIGURE 6.6 : Vue globale de la plateforme

6.10 Conclusion

Cette phase finalise le pipeline en permettant la visualisation et l'exploitation des résultats. Elle transforme un système technique en une véritable plateforme décisionnelle accessible aux utilisateurs.

Conclusion Générale

Ce projet a permis de concevoir et de mettre en œuvre une plateforme Big Data dédiée à l'analyse et à la prédiction des prix immobiliers, en s'appuyant sur le dataset Ames Housing. L'objectif principal était de transformer des données brutes en informations exploitables à forte valeur ajoutée, à travers un pipeline de traitement complet et structuré.

L'architecture proposée repose sur une approche modulaire et distribuée, intégrant plusieurs technologies complémentaires. La première phase a consisté en l'ingestion et le prétraitement des données à l'aide de scripts Python dans un environnement conteneurisé, garantissant la reproductibilité et l'isolation des traitements. Les données nettoyées ont ensuite été stockées dans un cluster HDFS, offrant une solution adaptée au stockage de volumes importants de données.

La phase suivante a permis l'exploitation de ces données à travers Apache Spark, en mettant en œuvre plusieurs modèles de machine learning pour la prédiction des prix immobiliers. L'évaluation comparative des modèles a permis d'identifier les approches les plus performantes, tout en assurant la traçabilité des résultats grâce à leur stockage dans une base de données relationnelle PostgreSQL.

Par ailleurs, un schéma de données normalisé a été conçu afin de structurer les informations métier, facilitant ainsi leur exploitation ultérieure. Enfin, une interface web a été développée pour visualiser les résultats sous forme de tableaux de bord interactifs, permettant aux utilisateurs d'explorer les données et d'interpréter les prédictions de manière intuitive.

Ce projet illustre ainsi l'intérêt des technologies Big Data dans le domaine immobilier, en montrant comment une chaîne de traitement bien conçue peut transformer des données complexes en outils d'aide à la décision. Il met également en évidence l'importance de l'intégration des différentes couches du système, depuis l'ingestion jusqu'à la visualisation.

En perspective, plusieurs améliorations peuvent être envisagées, notamment l'intégration de données en temps réel, l'optimisation des modèles de prédiction, ou encore le déploiement de la plateforme dans un environnement cloud pour une meilleure scalabilité.

Ainsi, ce travail constitue une base solide pour le développement de solutions intelligentes dans le domaine de l'analyse de données immobilières.

